

The Normalizing-Flow f_0 Method (Test 10): A Step-by-Step Guide

`fit_h_flow_f0_joint_mle` in `fitter.py`

July 2026

The idea in one sentence

We do not know the initial distribution function $f_0(z_0, w_0)$ of the stars, so we model it with a *normalizing flow* (a flexible, trainable probability density), and we fit this flow *jointly* with the dark-matter scale height h by maximizing the likelihood of the observed snapshot, where the two are connected through back-integration of the orbits and Liouville's theorem.

Contents

1	Setup: what we have and what we want	1
2	Step 1: The physics link — Liouville's theorem	1
3	Step 2: What a normalizing flow is	2
4	Step 3: The bijection — RealNVP affine coupling layers	2
5	Step 4: The Jacobian — why it costs nothing	3
6	Step 5: The flow <i>as</i> f_0 — assembling the loss	4
7	Step 6: Joint minimization — one optimizer, four parameter groups	4
8	Step 7: Why this fails — the Liouville degeneracy (the point of Test 10)	5
A	Appendix: mapping the math to the code	6

1 Setup: what we have and what we want

What we observe. A snapshot of N stars at time t_{obs} : positions and vertical velocities

$$\{(z_i, w_i)\}_{i=1}^N, \quad z \text{ in pc, } w \text{ in km/s.}$$

What we want. The dark-matter scale height h , which enters the vertical potential through

$$\Phi_{\text{DM}}(z) = 4\pi G \rho_{\text{DM}} h^2 \ln \cosh\left(\frac{z}{h}\right),$$

added to the fixed baryonic (thin disk, thick disk, gas) sech^2 components.

The catch. The likelihood of the data depends on the potential *and* on the (unknown) initial distribution function f_0 at $t = 0$. In earlier tests f_0 was assumed to be a Gaussian with fixed (possibly wrong) parameters. Test 10 asks: what if we instead let f_0 be a completely flexible learned density — a normalizing flow?

2 Step 1: The physics link — Liouville’s theorem

Stars move in phase space (z, w) under Hamilton’s equations,

$$\dot{z} = w, \quad \dot{w} = -\frac{\partial \Phi(z, t)}{\partial z}.$$

The collisionless Boltzmann equation says f is *constant along orbits*:

$$\frac{df}{dt} = 0 \implies f_{\text{obs}}(z, w) = f_0(z_0(z, w), w_0(z, w)),$$

where (z_0, w_0) is the point you land on if you take the observed star at (z, w) and integrate its orbit **backwards** from t_{obs} to $t = 0$ through the trial potential.

Why there is no Jacobian from the dynamics

In general, when you transform a density through a map you must multiply by the Jacobian determinant of the map. Here the back-integration map $(z, w) \mapsto (z_0, w_0)$ is a *Hamiltonian (symplectic)* map, and the code implements it with a leapfrog integrator, which is symplectic too. Symplectic maps preserve phase-space volume exactly:

$$\left| \det \frac{\partial(z_0, w_0)}{\partial(z, w)} \right| = 1.$$

Therefore the density of an observed star is *literally* f_0 evaluated at its back-integrated point — no correction factor. This is why the loss below is so simple.

So the log-likelihood of the snapshot, given a potential (parametrized by h , plus an optional neural-network correction $\delta\Phi$) and given f_0 , is

$$\log \mathcal{L} = \sum_{i=1}^N \log f_0(z_0(z_i, w_i; h, \delta\Phi), w_0(z_i, w_i; h, \delta\Phi)). \quad (1)$$

Everything that follows is about (a) how f_0 is represented by a flow, and (b) how (1) is maximized over the flow *and* h at the same time.

3 Step 2: What a normalizing flow is

A normalizing flow represents an arbitrary density by **deforming a simple one**. Take a base random variable $\mathbf{u} \in \mathbb{R}^2$ with a standard normal density

$$p_{\text{base}}(\mathbf{u}) = \frac{1}{2\pi} \exp\left(-\frac{1}{2}\|\mathbf{u}\|^2\right),$$

and an *invertible, differentiable* map T (the “bijection”) from base space to data space, $\mathbf{x} = T(\mathbf{u})$. The classical change-of-variables formula gives the density of $\mathbf{x}$:

$$\log p(\mathbf{x}) = \log p_{\text{base}}(T^{-1}(\mathbf{x})) + \log \left| \det \frac{\partial T^{-1}(\mathbf{x})}{\partial \mathbf{x}} \right|. \quad (2)$$

Read it as: push the data point back to base space, score it under the Gaussian, and add the log-Jacobian that accounts for how the map stretches or compresses volume.

Two requirements drive the whole architecture:

- (i) T^{-1} must be cheap to evaluate (we need it for every star, every epoch);
- (ii) the Jacobian determinant must be cheap (a general 2×2 determinant is cheap anyway, but the design below makes it *trivial* and generalizes to any dimension).

4 Step 3: The bijection — RealNVP affine coupling layers

The map T is a stack of K *affine coupling layers* (class `AffineCoupling`), each acting on the 2-D point. One layer works like this:

3.1 Split. Write the point as (a, b) : coordinate a is the *pass-through*, coordinate b is the one being *transformed*. A flag `flip` alternates the roles between layers (even layers transform w , odd layers transform z), so over the stack both coordinates get moved.

3.2 Condition. A small MLP takes the pass-through coordinate a and outputs two numbers: a log-scale $s(a)$ and a shift $t(a)$:

MLP: $1 \rightarrow 64 \rightarrow 64 \rightarrow 2$, $\tanh$ activations.

The log-scale is soft-bounded, $s = 3 \tanh(\hat{s}/3)$, so no single layer can stretch by more than $e^{\pm 3} \approx 20\times$ — a numerical-stability guard.

3.3 Affine transform. Only b changes:

$$\text{forward (data} \rightarrow \text{base, used in } \texttt{log_prob}): \quad u = (b - t(a)) e^{-s(a)}, \quad a \mapsto a, \quad (3)$$

$$\text{inverse (base} \rightarrow \text{data, used for sampling):} \quad b = u e^{s(a)} + t(a), \quad a \mapsto a. \quad (4)$$

Why this is invertible even though it contains a neural network

You never need to invert the MLP. Given the output, the pass-through coordinate a is right there unchanged, so you can recompute exactly the same $s(a)$ and $t(a)$ and undo the affine map in closed form. The network only *parametrizes* the affine map; the map itself is trivially invertible.

Identity initialization. The last linear layer of each MLP is initialized to zeros, so at the start $s = t = 0$ and every coupling layer is the identity. The flow therefore *begins* as an exact standard Gaussian and gradually deforms it during training. (This mirrors the zero-init convention used elsewhere in `fitter.py` — and recall the “dead-init” bug: here only the *last* layer is zero, the hidden weights are random, so gradients do flow.)

Standardization. Before entering the stack, physical coordinates are whitened with a *fixed* (non-trained) affine map,

$$\mathbf{x} = \left(\frac{z - \mu_z}{\sigma_z}, \frac{w - \mu_w}{\sigma_w} \right),$$

where μ, σ are the mean/std of the cloud back-integrated once at the initial guess h_{init} . This just puts the data at order unity for the network.

5 Step 4: The Jacobian — why it costs nothing

Take one coupling layer in the forward direction, mapping $(a, b) \rightarrow (a, u)$. Its Jacobian matrix is

$$\frac{\partial(a, u)}{\partial(a, b)} = \begin{pmatrix} 1 & 0 \\ \frac{\partial u}{\partial a} & e^{-s(a)} \end{pmatrix}.$$

The matrix is **triangular**: u depends on both inputs, but a depends only on itself. The determinant of a triangular matrix is the product of its diagonal, so the messy off-diagonal term (which involves derivatives of the neural network!) is irrelevant:

$$\boxed{\log \left| \det \frac{\partial(a, u)}{\partial(a, b)} \right| = -s(a)} \quad (5)$$

That single number per layer is exactly what `forward` returns alongside the transformed point. For the stack of K layers, log-determinants simply add:

$$\log |\det J_{\text{total}}| = \sum_{k=1}^K (-s_k) - \log(\sigma_z \sigma_w),$$

where the last term is the (constant) Jacobian of the whitening step — it converts the density back to *physical units*, per $\text{pc} \cdot \text{km/s}$, so it is directly comparable to a physical f_0 .

Putting Steps 2–4 together, `ZWNormalizingFlow.log_prob` computes:

$$\log f_0^{\text{flow}}(z_0, w_0) = \underbrace{-\frac{1}{2}\|\mathbf{u}\|^2 - \log 2\pi}_{\text{Gaussian base}} + \underbrace{\sum_{k=1}^K (-s_k)}_{\text{coupling Jacobians}} - \underbrace{\log(\sigma_z \sigma_w)}_{\text{whitening}}. \quad (6)$$

6 Step 5: The flow *as* f_0 — assembling the loss

Here is the conceptual switch of Test 9/10 relative to Test 4:

	Test 4	Tests 9/10
flow models...	the <i>observed</i> snapshot f_{obs}	the <i>initial</i> DF f_0
f_0 is...	known (double Gaussian)	unknown, learned
fit matches...	flow(z, w) to $f_0(z_0, w_0)$	data to flow via (1)

Substituting the flow (6) into the Liouville likelihood (1), the training loss for one epoch is

$$\mathcal{L}(\theta_{\text{flow}}, h, \theta_{\text{NN}}, A_{\text{NN}}) = -\frac{1}{|B|} \sum_{i \in B} \log f_0^{\text{flow}}\left(z_0(z_i, w_i; h, A_{\text{NN}} \delta\Phi_{\theta_{\text{NN}}}), w_0(\dots)\right), \quad (7)$$

where B is a random mini-batch of 500 stars. Note carefully what depends on what:

- h (and the optional NN potential) sit *inside the arguments*: they change *where* each star lands after back-integration;
- the flow parameters θ_{flow} change *how probable* any landing point is.

The back-integration `torch.back_integrate` is written entirely in PyTorch (1000 leapfrog steps), so automatic differentiation propagates gradients *through the whole orbit* into h : the optimizer literally feels “if h were slightly larger, star i would land at a slightly different (z_0, w_0) , where the flow assigns a different probability.”

The optional NN potential. When enabled, a time-dependent correction $A_{\text{NN}} \delta\Phi(z, t)$ is added to the force during back-integration. Its amplitude is hard-capped by projection so that its peak never exceeds 10% of the static potential’s peak (the `nn_frac_bound` cap): after every optimizer step, the peak of $|A_{\text{NN}} \delta\Phi|$ on a (z, t) grid is measured and A_{NN} is rescaled down if it exceeds the budget.

7 Step 6: Joint minimization — one optimizer, four parameter groups

A single Adam optimizer updates everything simultaneously, with per-group learning rates tuned to each parameter’s natural scale:

parameters	lr	role
flow weights θ_{flow}	10^{-3}	reshape f_0
h (a single scalar <code>nn.Parameter</code>)	2.0	move the potential
NN potential weights θ_{NN}	10^{-2}	shape of $\delta\Phi(z, t)$
amplitude A_{NN}	5×10^{-2}	size of $\delta\Phi$

One epoch (of 200) does:

1. draw a random subsample of 500 observed stars;
2. back-integrate them $t_{\text{obs}} \rightarrow 0$ with the *current* h and $\delta\Phi$ (differentiable leapfrog);
3. evaluate the loss (7) with the *current* flow;
4. backpropagate; clip the gradient on h (norm 50) and on the NN (norm 1);
5. Adam step on all groups at once;
6. clamp h into its bounds and project A_{NN} onto the 10% cap.

So the flow is chasing whatever cloud the current h produces, while h is simultaneously being pulled toward whatever makes the cloud most probable under the current flow.

8 Step 7: Why this fails — the Liouville degeneracy (the point of Test 10)

The trap

The same Liouville theorem that makes the loss elegant makes it *flat in h* . For (almost) *any* trial h , the back-integration produces *some* smooth cloud at $t = 0$ — and a sufficiently flexible flow can fit *that* cloud just as well as any other. Wrong- h clouds are just as “fittable” as the right one. The likelihood then barely prefers the true h , and nothing anchors it except the flow’s *limited capacity*.

Formally: if Ψ_h denotes the back-map at scale height h , then for any h the choice $f_0 = f_{\text{obs}} \circ \Psi_h^{-1}$ (the pushforward of the data) achieves the same maximal likelihood. A rigid parametric f_0 (a Gaussian) cannot follow this family, so it *breaks* the degeneracy — that is where the constraint on h actually comes from.

Test 10 measures this directly with the **init-spread** $|h(\text{init } 180) - h(\text{init } 350)|$: run the same fit from two starting guesses; if the likelihood really pins h , both must converge to the same value. The pre-run results ($N = \text{data size}$, flow size = couplings $\times$ hidden):

model of f_0	init-spread, $N=2000$	init-spread, $N=500$
rigid Gaussian (even with wrong σ_w)	~ 1 pc	~ 1 pc
small flow (8×64)	~ 3 pc	~ 12 pc
big flow (16×128)	~ 8 pc	~ 23 pc

More flexibility or less data $\Rightarrow$ weaker constraint, exactly as the degeneracy argument predicts. And the NN potential does not rescue the rigid-but-biased Gaussian either: from one init it stays idle, from the other it *destabilizes* h ($246 \rightarrow 297$).

Takeaway

The information about h in a snapshot-likelihood fit comes from *restricting* f_0 , not from the data alone. Relaxing f_0 into a flow destroys the constraint, and adding potential flexibility (the NN) cannot rebuild it. This dead-end is what motivates changing the *objective*: instead of a snapshot likelihood, use a statistic that directly targets the non-equilibrium term, e.g. a collisionless-Boltzmann residual $\partial f / \partial t$.

A Appendix: mapping the math to the code

math object	code (fitter.py)
coupling bijection, $\pm s$ log-det	<code>AffineCoupling.forward / .inverse</code>
soft bound $s = 3 \tanh(\hat{s}/3)$	<code>_scale_shift</code>
change of variables (6)	<code>ZWNormalizingFlow.log_prob</code>
whitening μ, σ	<code>set_standardization</code> (from h_{init} cloud)
back-map Ψ_h (symplectic leapfrog)	<code>torch.back_integrate</code>
joint loss (7)	<code>loss = -flow.log_prob(z0, w0).mean()</code>
joint fit driver	<code>fit_h_flow_f0_joint_mle</code>
10% cap projection	<code>_vpeak() + A_nn.mul_(A_nn_max/vp)</code>
Test 10 experiment	notebook cells <code>test10-md</code> , <code>test10-code</code>